Historical Week 2: Bulk Notification BackgroundService

> **Superseded implementation snapshot.** This document records how the Week 2 in-process `BackgroundService` requirement was originally met. Week 3 removed the in-memory channel/dictionary and API-hosted worker. The current system persists jobs in SQL Server, publishes through RabbitMQ, and processes them in the separate `NSA.Worker` executable. See `docs/adr/004-rabbitmq-worker-and-dlq.md` and `evidence/week-03/topology.md`. Paths and code excerpts below are historical and must not be used as current run instructions.

Requirement: Add a `BackgroundService`; `POST /notifications/bulk` returns `202 Accepted` plus a `jobId`; a status endpoint reflects progress.

> **Week 2 result:** At that milestone, the requirement was fulfilled by the controller, in-memory job service, API-hosted background worker, response DTO, and dependency-injection registration. The public 202/status contract remains, but ADR 004 supersedes the internal implementation for Week 3.

1. POST /notifications/bulk returns 202 and jobId

`Presentation/Controllers/Notification/NotificationsController.cs — lines 85–103`

```
[HttpPost("bulk")]
public ActionResult<BulkNotificationJobDto> CreateBulkNotifications(...)
{
    var job = bulkJobs.Queue(request);
    ...
    return Accepted(statusLocation, job);
}
```

`bulkJobs.Queue(request)` creates and queues the job. `Accepted(statusLocation, job)` produces HTTP 202, puts the polling URL in the `Location` header, and sends the job DTO in the response body.

The actual routes supported by the versioned controller are:

- `POST /api/v2/notifications/bulk`
- `POST /api/v1/notifications/bulk`
- `POST /api/notifications/bulk`

Response contract

Application/Contracts/Notification/BulkNotificationDtos.cs — lines 60–85

```
{
  "jobId": "...",
  "status": "Queued",
  "totalCount": 10,
  "processedCount": 0,
  "succeededCount": 0,
  "failedCount": 0,
  "queuedAtUtc": "...",
  "startedAtUtc": null,
  "completedAtUtc": null,
  "error": null
}
```

2. The job is queued for background processing

Service/Notification/BulkNotificationJobService.cs — lines 40–89

The `Queue()` method:

1. Rejects an empty batch.
2. Enforces the configured maximum batch size.
3. Creates a unique ID using `Guid.NewGuid()`.
4. Stores the job in a `ConcurrentDictionary`.
5. Writes the job into a bounded in-memory `Channel`.
6. Returns the initial job snapshot with status `Queued`.

```
HTTP request → queue job → return 202 immediately
                    ↓
  Background worker reads and processes the job
```

The bounded queue prevents unlimited memory growth. If the queue or tracked-job capacity is exhausted, the service throws an exception that is exposed as `503 Service Unavailable`.

3. A BackgroundService processes the job

`Infrastructure/BackgroundServices/BulkNotificationWorker.cs — lines 9–58`

```
public sealed class BulkNotificationWorker(...) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        await foreach (var job in jobs.ReadAllAsync(stoppingToken))
        {
            ...
        }
    }
}
```

The worker continuously reads jobs from the channel. For each job it:

1. Calls `job.Start()`, changing the status from `Queued` to `Processing`.
2. Loops through every notification.
3. Creates a dependency-injection scope for each item because database-backed services are scoped.
4. Uses `INotificationDispatcher` for email items.
5. Uses `INotificationService` for other channels.
6. Calls `RecordSuccess()` or `RecordFailure()`.
7. Calls `Complete()` after all items have been attempted.

An individual item failure is caught inside the loop. This means one bad notification does not prevent the rest of the batch from being processed.

Worker registration

`Program.cs — lines 109–116`

```
builder.Services.AddSingleton<BulkNotificationJobService>();
builder.Services.AddSingleton<IBulkNotificationJobService>(...);
builder.Services.AddHostedService<BulkNotificationWorker>();
```

The job service is a singleton so the HTTP controller and background worker share the same queue and job dictionary. `AddHostedService` starts the worker with the application.

4. The status endpoint reflects progress

`Presentation/Controllers/Notification/NotificationsController.cs — lines 105–116`

```
GET /api/v2/notifications/bulk/{jobId}

var job = bulkJobs.GetStatus(jobId);
return job is null ? NotFound() : Ok(job);
```

`GetStatus()` reads the job from the shared dictionary and returns a snapshot containing its current status and counters.

Method	Effect visible through status endpoint
`Start()`	Status becomes `Processing`; start time is recorded.
`RecordSuccess()`	Increments processed and succeeded counts.
`RecordFailure()`	Increments processed and failed counts; records a safe summary error.
`Complete()`	Status becomes `Completed` or `CompletedWithErrors`; completion time is recorded.
`Cancel()`	Status becomes `Cancelled`.

All state reads and writes use `lock (sync)`. Therefore, polling while the worker updates the job returns a consistent snapshot.

```
Queued → Processing (0/10) → Processing (4/10) → Completed (10/10)
```

Important limitation

> The queue and status dictionary exist only in application memory. Restarting the application loses queued jobs and their status. Completed jobs are also removed after the configured retention period. This is asynchronous processing, but it is not a durable job system.

Requirement-to-code map

Requirement	Implementation
Add a `BackgroundService`	`Infrastructure/BackgroundServices/BulkNotificationWorker.cs`
POST bulk returns 202 + jobId	`NotificationsController.CreateBulkNotifications()` and `BulkNotificationJobDto`

Requirement	Implementation
Queue work outside HTTP request	`BulkNotificationJobService` and its bounded `Channel`
Status endpoint reflects progress	`NotificationsController.GetBulkNotificationStatus()` plus synchronized job counters
Worker starts with application	`Program.cs: AddHostedService<BulkNotificationWorker>()`